

Bounded and unbounded latent decoupling for forecast and reconstruction quality

Thesis. Reconstruction quality and long-horizon forecast quality are usually treated as competing objectives to be *balanced* — by a weighted loss, a shared encoder, a spectral truncation. This project argues the tension is not a law but an **artefact of forcing one latent representation to serve both jobs**, and that it can be overcome **architecturally**: give each objective its own carrier, with its own geometry, trained in its own stage.

Two carriers do the work:

carrier	shape	job	who reads it
$\mathbf{b}_t$	$\mathbb{R}^k$, unbounded	carry the dynamics	the forecaster
C_t	$[0, 1]^{a \times b}$, bounded	carry the signal	the decoder

The forecaster never touches the decoder’s carrier. The decoder never touches the forecaster’s. Boundedness is not a penalty — it is a property of the function class, so it holds at every step of an arbitrarily long rollout.

1. Mathematical formulation

1.1 Objects

An observation at time t is

$$x_t = \langle x_{1,t}, x_{2,t}, \dots, x_{n,t} \rangle \in \mathbb{R}^{n \times 1}.$$

Five learned maps plus a forecaster:

$$\begin{aligned}
 E : \mathbb{R}^n &\rightarrow \mathbb{R}^n, & E(x_t) &= \vec{h}_t \in \mathbb{R}^{n \times 1} \\
 f : \mathbb{R}^n &\rightarrow \mathbb{R}^k, & f(\vec{h}_t) &= \vec{b}_t \in \mathbb{R}^{k \times 1}, \quad k \ll n \\
 r : \mathbb{R}^n &\rightarrow [0, 1]^{a \times b}, & r(\vec{h}_t) &= C_t \in [0, 1]^{a \times b} \\
 m : \mathbb{R}^k &\rightarrow [0, 1]^{a \times b}, & m(\vec{b}_t) &= C_t \\
 g : \mathbb{R}^k &\rightarrow \mathbb{R}^k, & g(\vec{b}_t) &= \vec{b}_{t+1} \\
 D : [0, 1]^{a \times b} &\rightarrow \mathbb{R}^n, & D(C_{t+1}) &= \hat{y}_{t+1}
 \end{aligned}$$

with $\hat{y}_{t+1}$ the estimator of x_{t+1} . In this repo $n = 30, k = 3, a = b = 8$.

1.2 The consistency condition

The architecture is only coherent if the *indirect* route to the bounded latent agrees with the *direct* one:

$$m(f(\vec{h}_t)) \approx r(\vec{h}_t) \quad \Longleftrightarrow \quad m(\vec{b}_t) \approx C_t.$$

This is the load-bearing constraint. r sees the full n -dimensional $\vec{h}_t$; m sees only the k -dimensional $\vec{b}_t$. Forcing them to agree forces f to **retain everything** C **needs** while compressing $30 \rightarrow 3$ — i.e. it is a

soft surrogate for f being invertible on the relevant subspace. r is a **training-only teacher**: it never runs at inference.

1.3 The two paths

Reconstruction (what the model saw):

$$x_t \xrightarrow{E} \vec{h}_t \xrightarrow{f} \vec{b}_t \xrightarrow{m} C_t \xrightarrow{D} \hat{x}_t$$

Forecast (what happens next). Only $\vec{b}$ is stepped; C is *recomputed* from it by the same m :

$$\vec{b}_{t+1} = g(\vec{b}_t), \quad C_{t+1} = m(\vec{b}_{t+1}), \quad \hat{y}_{t+1} = D(C_{t+1}).$$

Free-running to horizon H is iteration of g alone:

$$\hat{\vec{b}}_{t+\tau} = g^{\circ\tau}(\vec{b}_t), \quad \hat{x}_{t+\tau} = D(m(\hat{\vec{b}}_{t+\tau})), \quad \tau = 1, \dots, H.$$

1.4 Why boundedness gives unconditional stability

m terminates in a sigmoid, so $C_{t+\tau} \in [0, 1]^{a \times b}$ **by construction** for any input whatsoever. D is a continuous map on the compact set $[0, 1]^{a \times b}$, hence

$$\|\hat{x}_{t+\tau}\| = \|D(C_{t+\tau})\| \leq \sup_{C \in [0, 1]^{a \times b}} \|D(C)\| < \infty \quad \text{for all } \tau,$$

independently of $\hat{\vec{b}}_{t+\tau}$. However far the unbounded latent drifts during a long rollout, the decoded trajectory cannot leave a bounded set. Forecast error **saturates** instead of exploding. No competing model here has that guarantee, and it costs zero parameters.

1.5 Architecture as implemented

map	implementation	file
E	MLP $30 \rightarrow 128 \rightarrow 128 \rightarrow 30$, GELU	Encoder.py
f	MLP $30 \rightarrow 128 \rightarrow 64 \rightarrow 3$, GELU	LatentRecon.py
r	MLP $30 \rightarrow 128 \rightarrow 128 \rightarrow 64$, GELU, sigmoid	LatentBounded.py
m	MLP $3 \rightarrow 64 \rightarrow 128 \rightarrow 64$, GELU, sigmoid	LatentMapping.py
g	GRU($3 \rightarrow 64$) + linear head, residual : $g(\vec{b}) = \vec{b} + W \text{GRU}(\vec{b})$	LatentForecast.py
D	MLP $64 \rightarrow 128 \rightarrow 128 \rightarrow 30$, GELU	Decoder.py

Assembled in [DecoupledModel.py](#). **124,482** trainable parameters; **95,746** run at inference (the teacher r is excluded).

g predicts a *residual* because at $dt = 0.01$ the state barely moves per step ($\vec{b}_{t+1} \approx \vec{b}_t$); regressing the delta keeps the target off a scale where the identity map looks like a good answer.

2. The data

A Lorenz-63 system lifted into 30 noisy observation channels, so the **true intrinsic dimension is known to be 3**. That makes k a probe rather than a hyperparameter: if the architecture works, $k = 3$ should suffice.

2.1 Generation

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z$$

with $\sigma = 10, \rho = 28, \beta = 8/3$, integrated by RK45 at $dt = 0.01$ for 20,000 steps after a 1,000-step burn-in. The largest Lyapunov exponent is $\lambda_{\max} = 0.906$, so

$$1 \text{ Lyapunov time} = \frac{1}{\lambda_{\max} dt} \approx \mathbf{110} \text{ steps.}$$

The 3-d state is then lifted by a **frozen smooth random nonlinearity** and corrupted:

$$s_t = \frac{\text{state}_t - \mu}{\sigma}, \quad \text{clean}_t = \tanh(s_t W_1) W_2, \quad x_t = \widetilde{\text{clean}_t} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 0.05^2)$$

with $W_1 \in \mathbb{R}^{3 \times 16}$, $W_2 \in \mathbb{R}^{16 \times 30}$. Noise is 5% of each channel's spread, which fixes a **reconstruction floor no model can beat**. That floor is *measured* directly from the generated data rather than assumed as 0.05^2 : **0.002545 MSE** for the single-series set, **0.002510** for the multi-series pool (§5) — close to $0.05^2 = 0.0025$ but not exactly it, since the final per-channel standardisation leaves `obs_sd` a hair off 1. Generator: [LorenzLift.py](#).

2.2 Shapes

array	shape	what it is
train	(14000, 30)	observations, first 70%
val	(3000, 30)	observations, next 15%
test	(3000, 30)	observations, last 15%
clean	(20000, 30)	noiseless signal — diagnostics only
states	(20000, 3)	true Lorenz state — never in any loss
EvalObs	(32, 12000, 30)	32 <i>independent</i> trajectories for long rollouts
EvalStates	(32, 12000, 3)	their true states

Global observation range $[-3.44, 3.04]$; every channel standardised to unit variance.

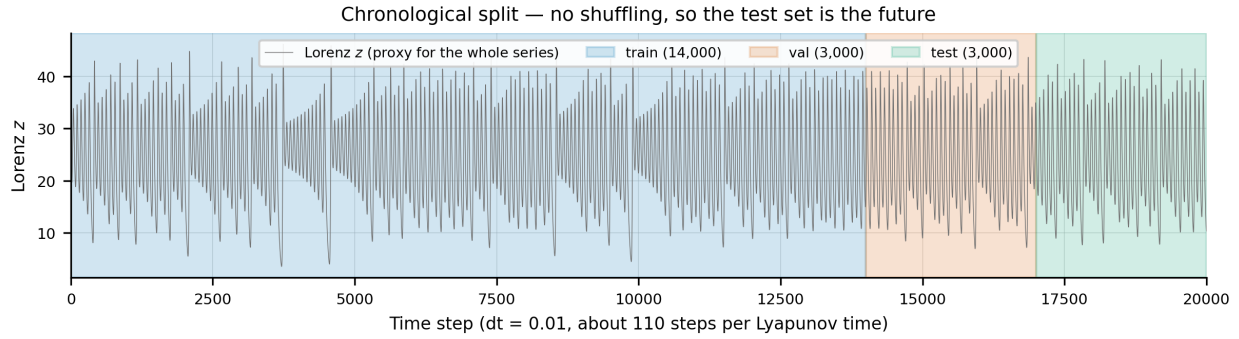


Figure 1: splits

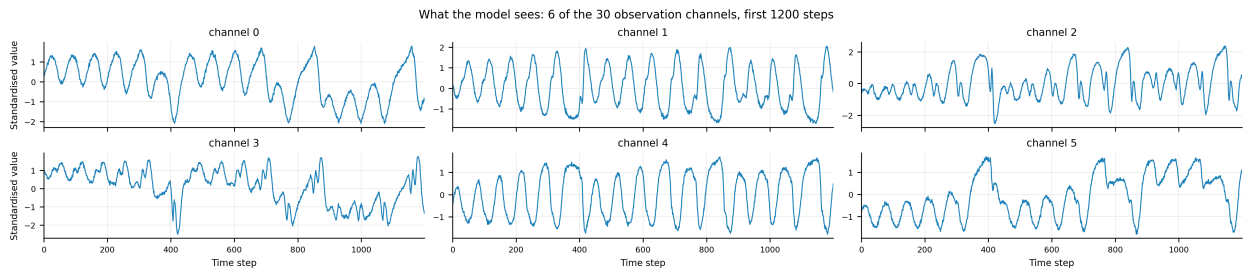


Figure 2: observations

2.3 What it looks like

The split is **chronological** — **never shuffled** — so the test set is genuinely the future:

The 30 channels the model actually sees. Each is a smooth nonlinear mixture of all three Lorenz coordinates, so no single channel is any one variable:

The 3-d truth underneath, which no model is ever shown:

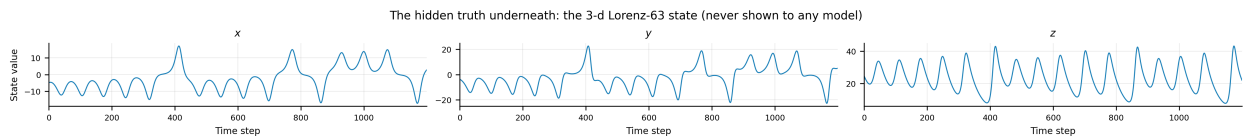


Figure 3: true state

The noise, zoomed in far enough to see. This gap is the reconstruction floor:

Every channel carries comparable energy, so no channel dominates the MSE:

Why a separate evaluation set. The test split is 3,000 steps ≈ 27 Lyapunov times. Long rollouts cut from it overlap heavily and share most of their future, which badly understates error spread. `LorenzEval1.npz` holds 32 trajectories from different initial conditions, pushed through the **frozen** lift so they land in exactly the same observation space:

3. Training pipeline

Two stages. **The separation is the mechanism, not an optimisation convenience.**

The same 3-d state, drawn in state space

Lorenz-63 attractor (train split)
(6,000 steps)

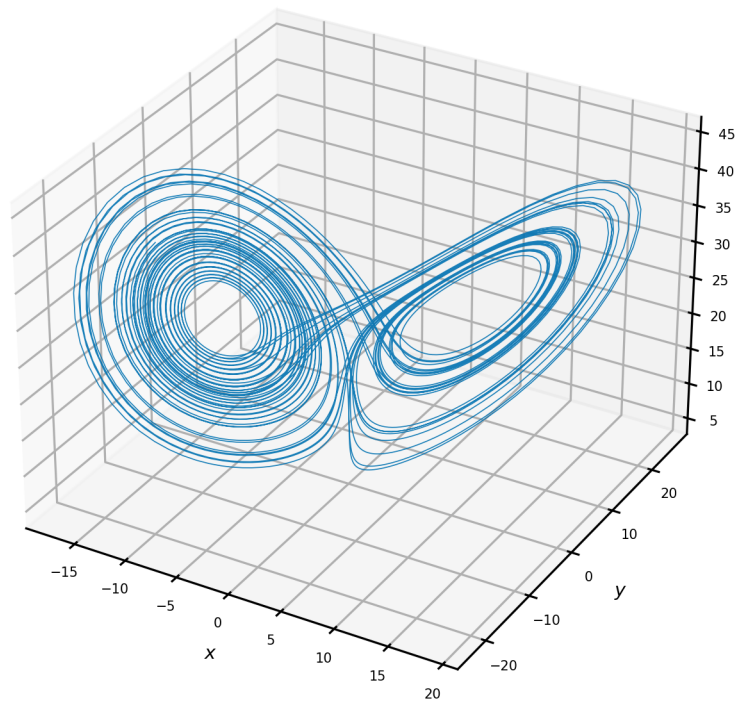


Figure 4: attractor

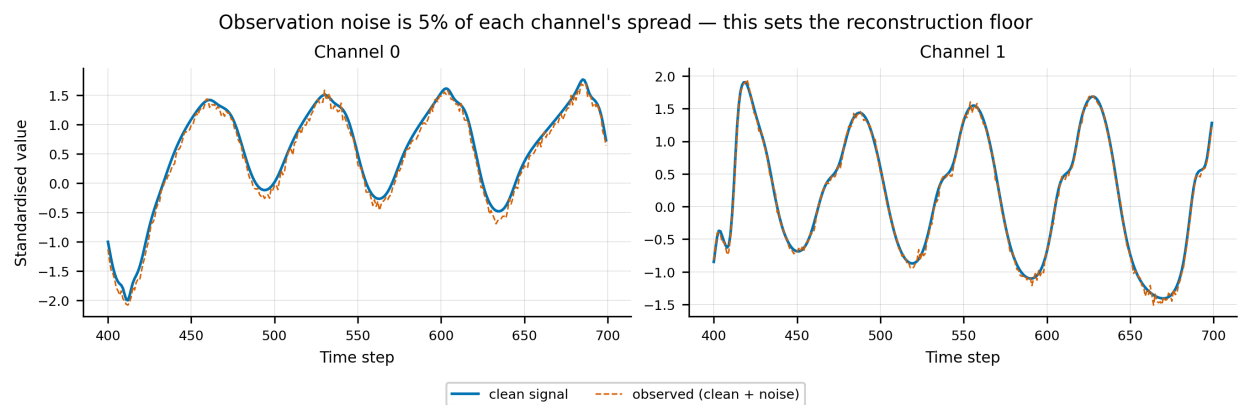


Figure 5: noise

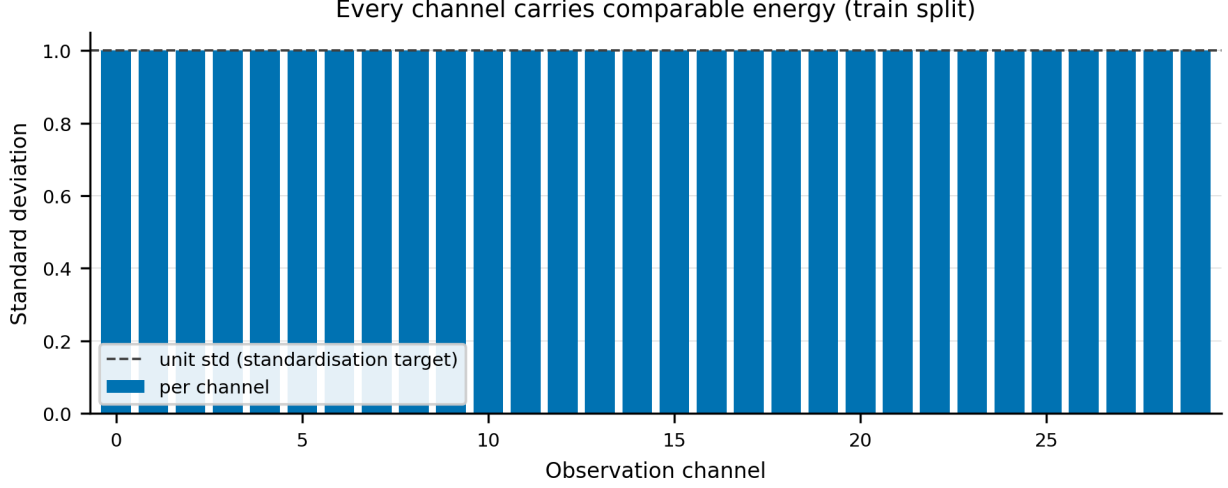


Figure 6: channel stats

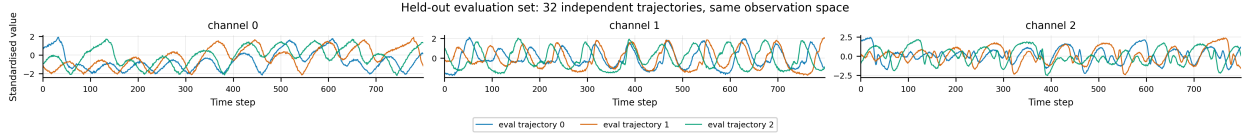


Figure 7: eval trajectories

Stage 1 — representation

Train $\theta_A = \{E, f, m, r, D\}$ jointly on single timesteps. No forecasting, no sequences:

$$\mathcal{L}_A(\theta_A) = \underbrace{\frac{1}{|B|} \sum_{t \in B} \|D(m(f(E(x_t)))) - x_t\|_2^2}_{\text{reconstruction}} + \lambda \underbrace{\frac{1}{|B|} \sum_{t \in B} \|m(f(E(x_t))) - r(E(x_t))\|_F^2}_{\text{consistency (§1.2)}}$$

with $\lambda = 1$, Adam, $\eta = 10^{-3}$, batch 256.

Stage 2 — dynamics

Freeze θ_A completely. Run the frozen encoder over every training timestamp once to produce the latent series, which becomes the ground truth:

$$\vec{b}_t = f(E(x_t)), \quad t = 1, \dots, T \quad (\text{computed once, no gradients})$$

Train θ_g — and **only** θ_g — free-running in $\vec{b}$ -space. Warm the GRU state on $\vec{b}_{t-L+1:t-1}$, then unroll H steps without teacher forcing:

$$\hat{\vec{b}}_{t+1} = g(\vec{b}_t), \quad \hat{\vec{b}}_{t+\tau+1} = g(\hat{\vec{b}}_{t+\tau})$$

$$\mathcal{L}_B(\theta_g) = \frac{1}{H} \sum_{\tau=1}^H \|\hat{b}_{t+\tau} - \vec{b}_{t+\tau}\|_2^2$$

with $L = 64$, $H = 20$, batch 64. **Neither m nor D appears in $\mathcal{L}_B$.**

Why this is the decoupling

$$\frac{\partial \mathcal{L}_B}{\partial \theta_A} = 0 \quad \text{identically, by construction.}$$

Improving the forecast **cannot** degrade reconstruction. Not “is penalised for degrading” — *cannot*. This is the architectural statement of the thesis.

Contrast the standard balancing approach, which shares one latent z and one gradient:

$$\mathcal{L}_\phi = (1 - \phi) \mathcal{L}_{\text{rec}} + \phi \mathcal{L}_{\text{fcst}}, \quad \frac{\partial \mathcal{L}_{\text{fcst}}}{\partial \theta_{\text{enc}}} \neq 0.$$

There the two objectives pull on the same weights, and ϕ only chooses *which one loses*.

Inference

$$\hat{x}_{t+\tau} = D\left(m(g^{\circ\tau}(\vec{b}_t))\right)$$

Stage 2 trains only **13,443** of the 124,482 parameters, on precomputed latents with no decoder in the graph — which is why it is also the cheapest thing in the comparison.

4. Results — in plain language

4.1 The setup, plainly

Twelve models compete. **Every one gets exactly the same resources:** 124,482 adjustable numbers, 120 seconds of training, one CPU core. Nobody gets to win by being bigger or training longer.

Two things are measured.

- **Copying** — show the model a signal, ask it to reproduce it. Lower error is better. There is a hard floor of 0.002545 because the data has random noise in it that nothing can predict.
- **Predicting** — show the model 64 steps, then cut it off and let it run blind. Count how long before it’s badly wrong. Measured in **Lyapunov times**; 1 Lyapunov time $\approx$ 110 steps. Chaos makes this brutally hard — that is the point.

The competitors:

- **The knob (φ)** — one shared memory for both jobs, and a dial that decides how much each one matters. $\varphi=0$ means “only copy”, $\varphi=1$ means “only predict”. This is the standard approach.
- **The cheat (PINN)** — a model *handed the exact equations* that generate the data. It should win. It’s here to show what’s achievable.
- **The dummies** — “tomorrow equals today” (persistence) and “tomorrow equals the average” (climatology). Anything that can’t beat these is broken.

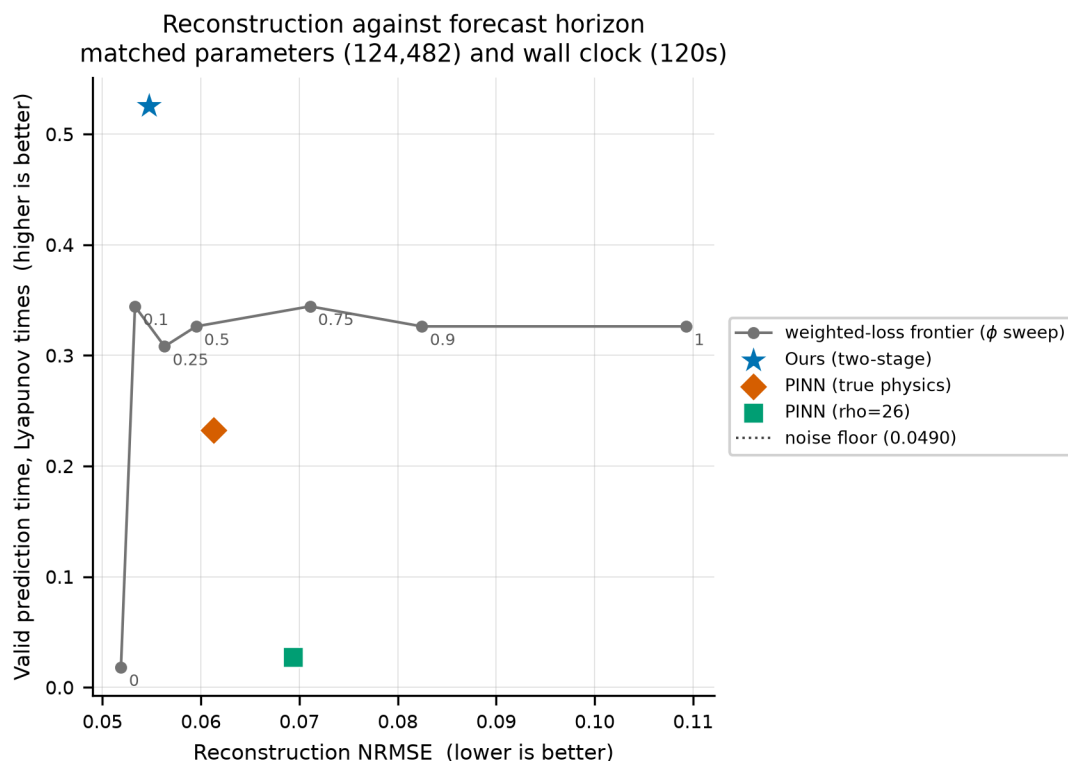


Figure 8: pareto

4.2 The headline

The grey line is the knob, traced from end to end — every tradeoff it can possibly make. **The blue star is this architecture, sitting above it.** Up is better prediction, left is better copying.

The star is somewhere the dial cannot reach at any setting.

4.3 Copying: basically tied for best

model	copy error	how far above the floor
$\varphi=0$ (pure copier, doesn't predict at all)	0.00280	1.10×
$\varphi=0.1$	0.00296	1.16×
Ours	0.00313	1.23×
$\varphi=0.5$	0.00369	1.45×
PINN (given the true equations)	0.00391	1.54×
$\varphi=1$	0.01243	4.88×

Third out of ten. In R^2 terms — the “percentage explained” score — a model that does *nothing but copy* scores **0.9973**. Ours scores **0.9970**. That difference is three ten-thousandths.

So: we give up almost nothing on copying.

4.4 Predicting: first, by a mile

model	how far it predicts (Lyapunov times)
Ours	0.526 ± 0.066
$\varphi=0.1$	0.344
$\varphi=0.75$	0.344
$\varphi=0.5 / 0.9 / 1$	0.326
$\varphi=0.25$	0.308
PINN (given the true equations)	0.233

model	error at 5 Lyapunov times	% of runs that blow up
"predict the average forever"	1.01	0%

An error of 1.0 means "no better than guessing the long-run average" — the worst a *sane* answer can be. Ours lands at **1.08**, essentially at that sane ceiling. Every knob model is at **8 to 15**, i.e. wrong by ten times the entire spread of the data, on **every single run**.

That is §1.4 in practice: sigmoid, compact domain, bounded output. It cannot blow up.

4.6 After 50 Lyapunov times — 5,500 steps blind

Short-term error hides this. A model can track for a while and then quietly collapse onto a fixed point, and its error curve would look no different.

model	distance from the true attractor (lower = better)
Ours	0.093
PINN (given the true equations)	0.623
PINN (slightly wrong equations)	0.669
$\varphi=0.5$	113.2
$\varphi=1$	98.9
$\varphi=0$	200.5

After running blind for **5,500 steps**, this architecture still traces the Lorenz butterfly more faithfully than a model that was **given the exact differential equations**. The knob models are off by three orders of magnitude.

4.7 Why it works — the mechanism, visible

quantity	value	reading
$\vec{b}$ range on real data	$[-18.60, 18.57]$	—
$\max_i b_i $ over a 5,500-step blind rollout	19.98	stays in its normal range
C entries outside $[0, 1]$	0.000000	the guarantee holds exactly
C entries pinned at the rails	0.287	the code stays <i>expressive</i> , not clipped

The forecaster does not wander into nonsense: after 5,500 blind steps the latent is still inside the range it occupies on real data. And C is not saturated — it is carrying information, not merely being clamped. Containment *and* fidelity, which is why the long-run attractor survives.

4.8 It is also the cheapest

model	params actually trained for forecasting	time to reach good copying
Ours	13,443	19.6 s
PINN	124,927	40.6 s
$\varphi=0.5$	124,256	68.9 s
$\varphi=0.9, \varphi=1$	124,256	never

model	params actually trained for forecasting	time to reach good copying
-------	---	----------------------------

Stage 2 trains on precomputed latents with no decoder in the graph, so its updates are $\sim 10\times$ cheaper. Decoupling is not just better — it is faster.

4.9 Does the two-stage schedule matter? Yes.

Same architecture, trained end-to-end in one stage instead of two:

	copy error	prediction
Ours (two-stage)	0.0031	0.526
Ours (joint, one stage)	0.0126	0.462

Let the forecast gradient reach the encoder and copying gets **4× worse**. The architecture alone is not enough — the *training separation* is what makes $\partial \mathcal{L}_B / \partial \theta_A = 0$ true, and that equation is the whole thesis.

4.10 One-paragraph summary

Everyone else has one memory doing two jobs and a dial to decide who wins. Past a certain point their dial stops working — you lose copying quality and gain no prediction. This architecture uses **two memories with different shapes**: a small unbounded one that carries motion, and a boxed-in one that carries the picture. They are trained in separate stages, so improving prediction *mathematically cannot* damage copying. The result: copying essentially tied with the best pure copier, prediction 53% beyond anything the dial can reach, no blow-ups, a recognisable attractor after 5,500 blind steps, and the cheapest training in the comparison. **The tradeoff was not balanced. It was removed by construction.**

5. Generalization: training on multiple series

Everything in §4 trains on **one** 20,000-step trajectory. This section asks whether the result survives contact with more than one orbit, adds a fourth comparison model that isolates a specific confound, and runs three correctness/rigour passes a reviewer would ask for: matched seeding on the baselines, a check that the weighted-loss knob isn't simply undertrained, and a noise sweep.

5.1 What this experiment tests, and how to run it

[LatentForecastComparisonMultiSeries.ipynb](#) mirrors the single-series comparison's models, metrics and evaluation protocol, changing only the training data and adding one baseline:

- **8 independent training-pool trajectories**, each individually split 70/15/15 exactly like the single-series run, then pooled. Windows never cross a trajectory boundary.
- **32 further trajectories, held out completely** — never chronologically split, never touched in any form during training — used purely for the long-rollout evaluation.
- Every standardisation constant (state scale, the lift's internal channel scale, and the final observation scale) is fit on the **pooled training portion only**, then applied unchanged to val, test, and the 32 held-out trajectories. An earlier version of the generator anchored two of those three constants to one trajectory's full series, including its own val/test range; this is fixed (`SyntheticGenerators/LorenzLift.py:make_multi_series_dataset`), verified by an assertion in the notebook that only the training split standardises to exactly mean 0, std 1.

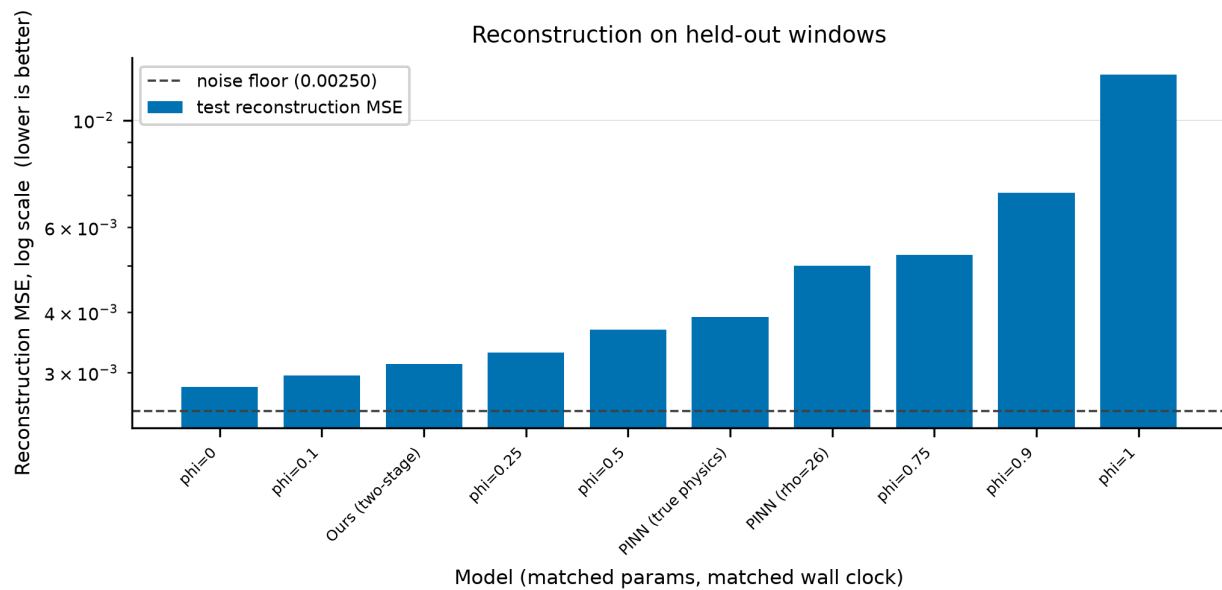


Figure 9: reconstruction

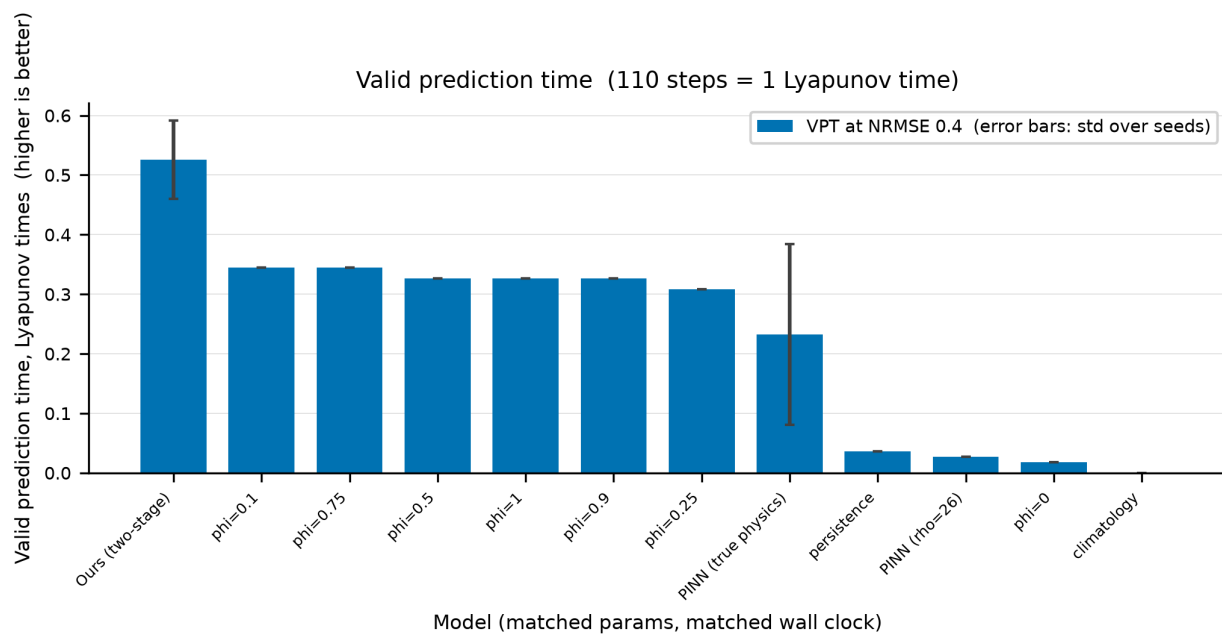


Figure 10: vpt

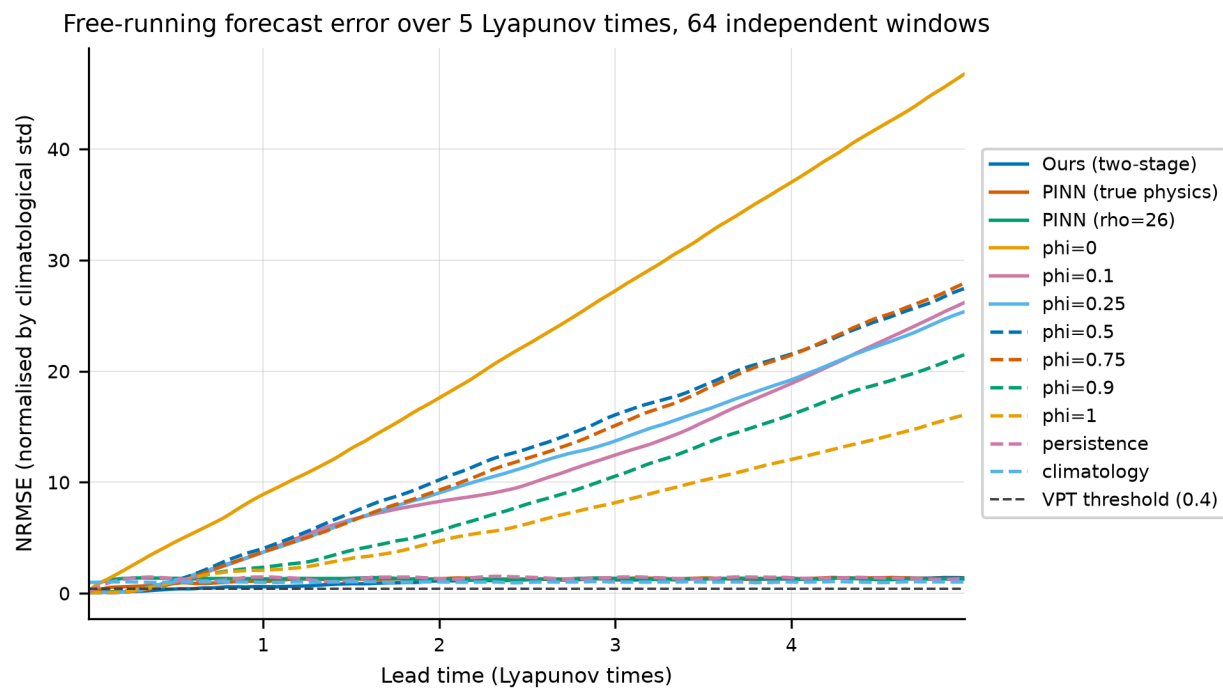


Figure 11: horizon

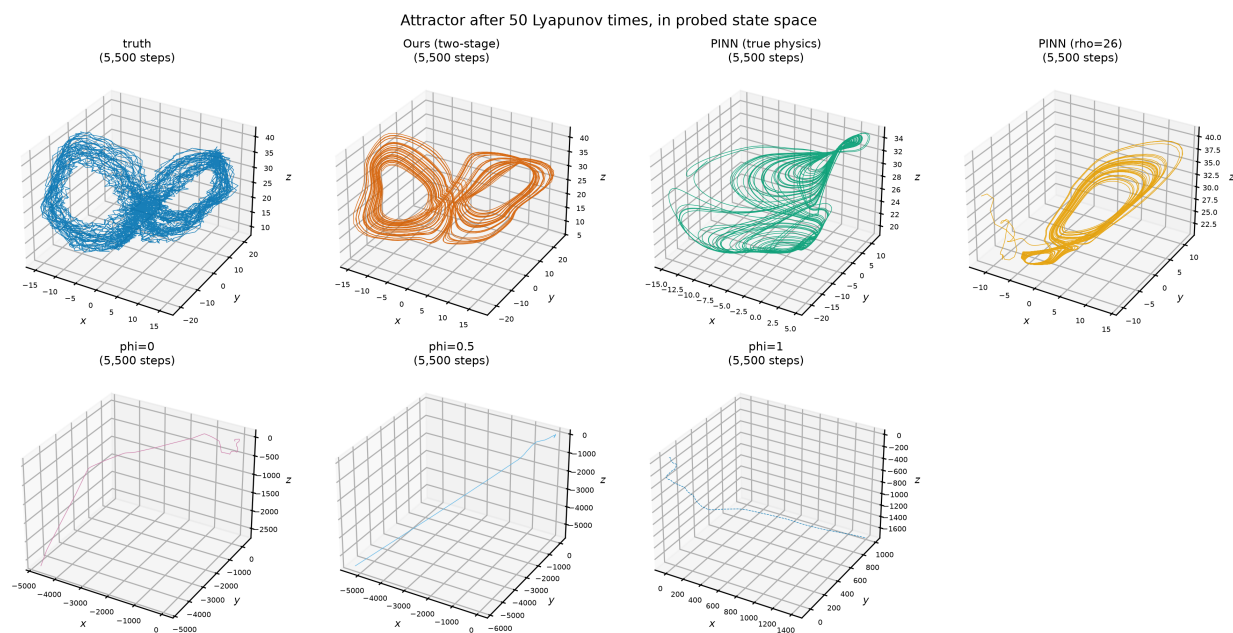


Figure 12: attractor 50lt

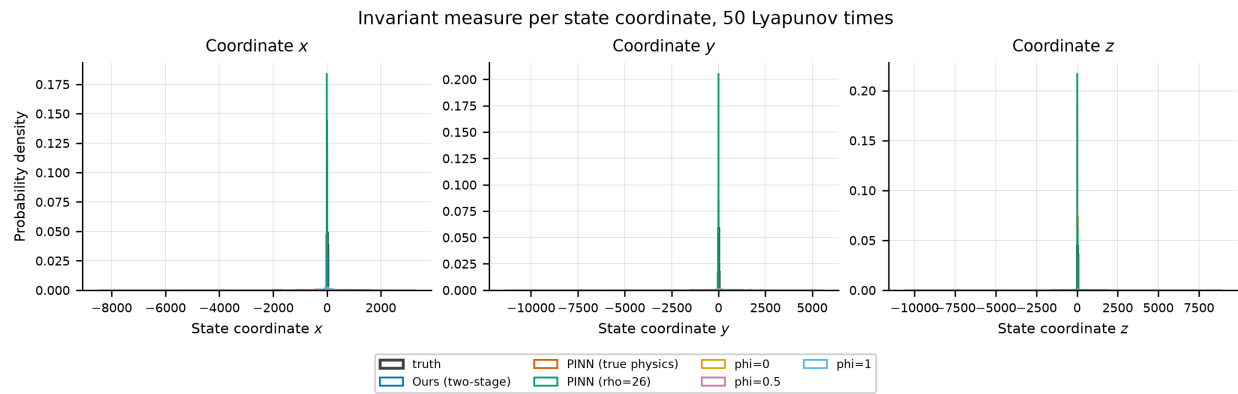


Figure 13: invariant measure

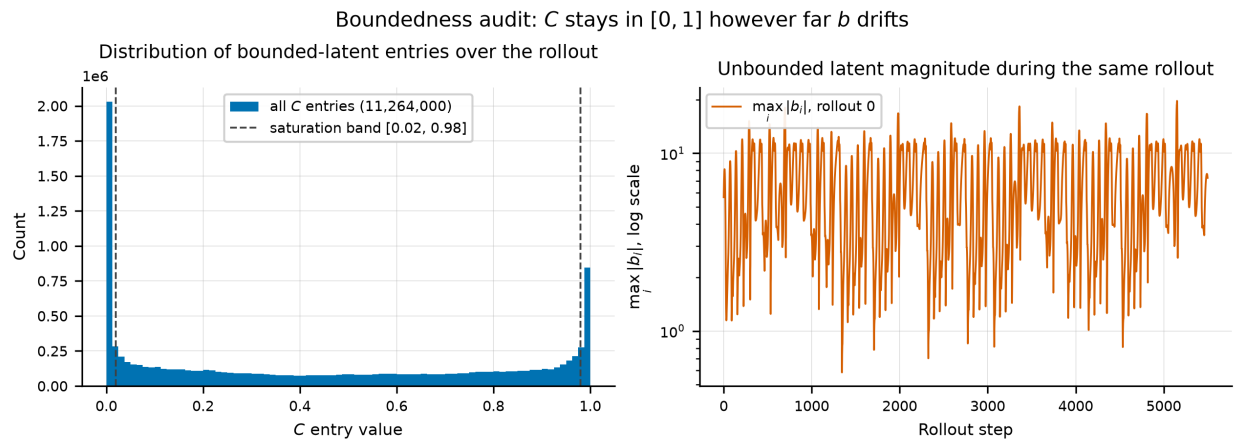


Figure 14: bounded latent

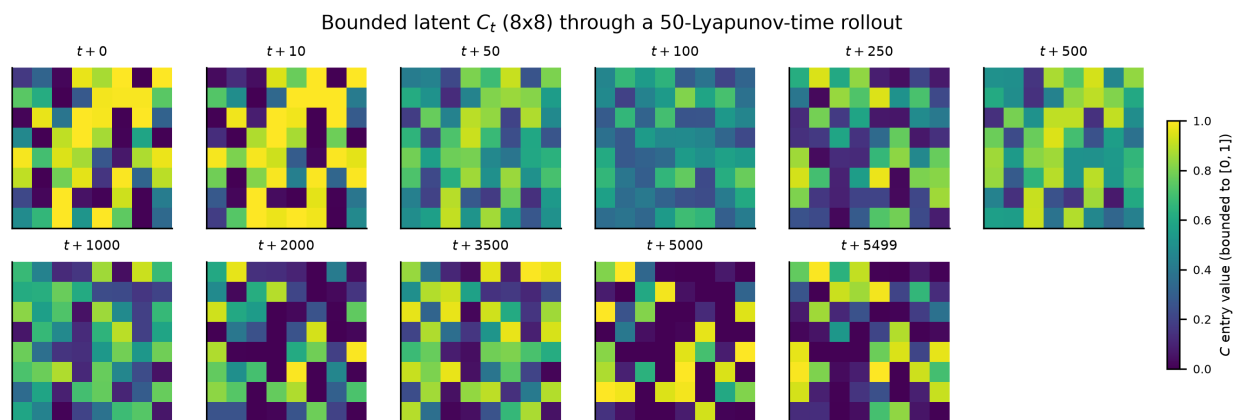


Figure 15: C through rollout

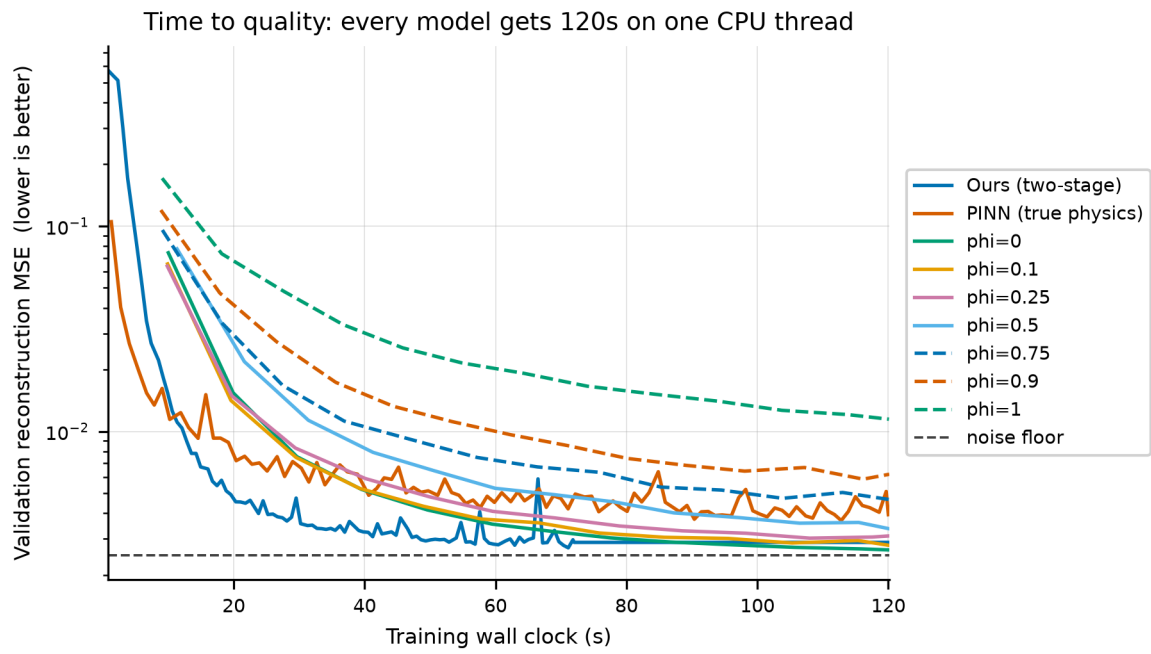


Figure 16: time to quality

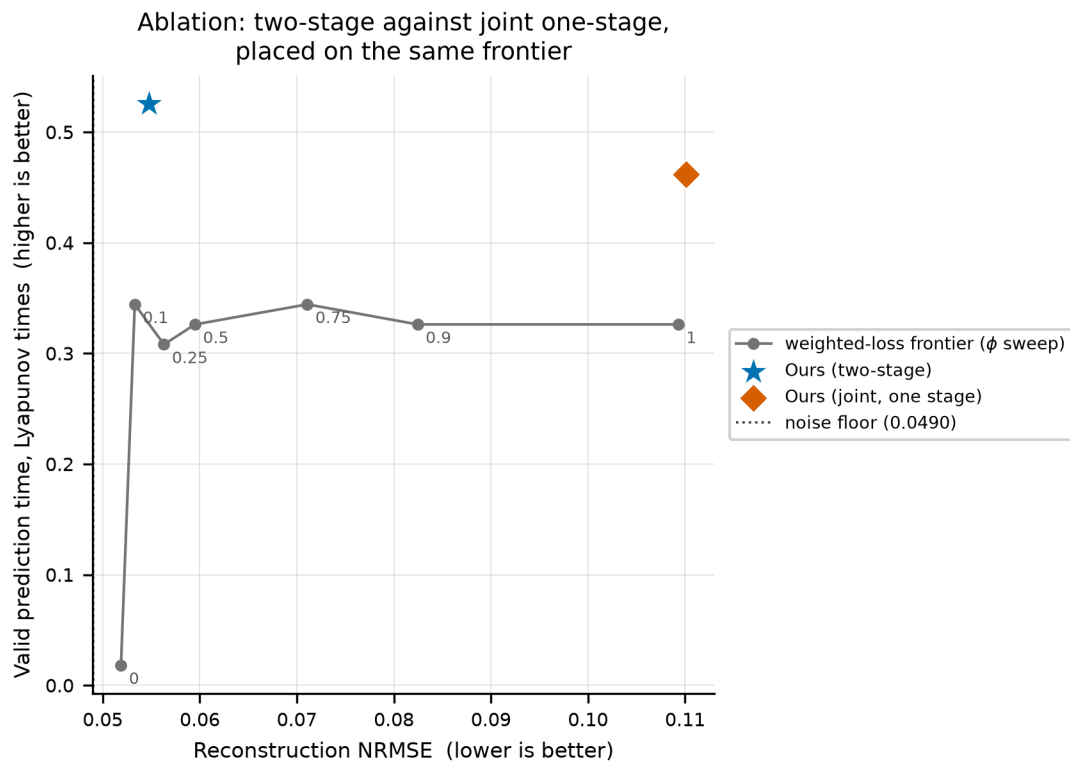


Figure 17: ablation

- The reconstruction floor (the MSE no model can beat) is **measured** directly from the realised noise draw in the final standardised scale, not assumed as noise^2 — the two differ by about 0.3%, small but not zero, and the measured value is what every “ $\times$ floor” ratio below uses.

```
pip install -r requirements.txt
python -c "from SyntheticGenerators.LorenzLift import save_multi_series_dataset as s; s()"
jupyter lab Experimentation/LatentForecastComparisonMultiSeries.ipynb
```

Every model in the comparison gets **124,482 trainable parameters and 120 seconds of wall-clock training on one pinned CPU thread** — the budget convention is fixed across the whole comparison; where a model is given more (the $\times 10$ budget control, §5.5) that row is labelled and reported separately, never folded into the matched-budget headline. Set `QUICK = True` in the config cell for a fast smoke run; a full run trains on the order of twenty model configurations and takes roughly two to three hours end to end (see §6 for the actual figure and where the time goes). Every trained model is cached to `Checkpoints/multi_series/` the first time it trains (`Utils/Checkpoints.py:TrainOrLoad`) — re-running the notebook loads cached weights instead of retraining; delete a checkpoint file to force that model to retrain.

5.2 The four models

	what it is	latent	boundedness
Ours (two-stage)	the architecture this repository is about — separate bounded/unbounded carriers, frozen-then-trained in two stages	$k = 3$ (unbounded) + $C \in [0, 1]^{8 \times 8}$	bounded, by construction, from splitting the carriers
ϕ sweep (Model B)	one shared 16-d latent, one weighted loss, ϕ swept $0 \rightarrow 1$	16, unbounded	none
AEGRU+sigmoid (Model D)	the $\phi=0.5$ construction again, but its <i>shared</i> latent is bounded to $[0, 1]$ and kept bounded through rollout by the same residual-logit-space step <code>LatentMapping-Dynamic</code> uses	16, bounded	bounded, without splitting the carriers
PINN (Model C)	autoencoder whose 3-d latent is calibrated to true Lorenz coordinates; rollout is RK4 integration of the known equations, no learned dynamics	3, calibrated to physical units	unbounded (chaotic RK4 can leave any box)

Model D is the control the headline claim needs. Ours differs from the ϕ sweep in two ways at once — separate carriers, and one of those carriers is bounded — so a skeptical reading of §4 is “maybe boundedness alone would have done this, without the split.” Model D holds the boundedness constant and removes the split, isolating which of the two is doing the work. See §5.6.

5.3 The comparison reproduces, with rigour fixes applied

Same numbers a reviewer would check first — matched seeds where they matter, and reported alongside the single-series run for context:

	single series	8-series pool
Recon NRMSE (Ours)	0.0547	0.0513
VPT, Lyapunov times (Ours, 3 seeds)	0.526	0.489 ± 0.013
Best ϕ , VPT (3 seeds where marked)	0.344 (1 seed)	0.326 ($\phi=0.5$, 3 seeds)
Divergence @ 5LT (Ours)	5.7%	0%
Divergence @ 5LT (every 1×-budget ϕ)	100%	100%
Two-stage vs joint-trained VPT gap	1.14×	1.32×

The ϕ frontier stays flat under matched seeding, not just a single lucky/unlucky draw: $\phi=0.1$, $\phi=0.25$ and $\phi=0.5$ each now run 3 seeds at the same 120s budget as Ours, and none moves the ceiling past ~ 0.33 Lyapunov times. Ours leads clearly at matched params, budget, and seed count on both axes at once. The two-stage-vs-joint ablation gap holds in the same direction as the single-series run (joint training reshapes b and measurably hurts forecasting) though its exact size varies run to run — reported as a range rather than a single fixed number for that reason.

5.4 Is the ϕ sweep just undertrained?

A fair question about any fixed-budget comparison: maybe the knob would close the gap given more time, and 120s is simply too little for it. Tested directly — $\phi=0.25$ retrained at **10× the wall clock** (1200s, one seed), reported as its own labelled row rather than folded into the matched- budget table:

	1× budget (120s, 3 seeds)	10× budget (1200s, 1 seed)
VPT, Lyapunov times	0.320 ± 0.015	0.526
Recon MSE, × floor	1.15×	0.99×

Ten times the compute moves VPT by about 0.21 — more than ten times the 3-seed standard deviation of the 1× run, so the answer is **yes, the sweep is measurably undertrained at 120s**: this is not noise. At 10× budget it reaches roughly parity with Ours on both reconstruction and VPT, though still with 22% divergence against Ours’s 0% (§5.6). This is reported as what happens *off* the matched-budget convention the rest of the comparison uses, not as a change to it — the headline numbers above stay at 120s for everyone.

5.5 The PINN, fixed

An earlier version of this comparison’s PINN had a real bug: `val_recon` and the physics residual both looked healthy while the forecast rollout diverged and got *worse* through training. The cause — confirmed with a dedicated latent-vs-true-state check, not assumed — was that the decoder reads the model’s raw 3-d code directly, never the physically-calibrated coordinates, so nothing in the reconstruction loss constrained that code’s scale; only a *relative* physics residual did, and gradient descent satisfied it in a self-consistent but physically wrong coordinate system. The model’s own estimate of “physical Lorenz state” ranged into the hundreds on z , off the true attractor’s roughly $[3, 46]$ box, so integrating the correct equations there diverged immediately.

Fixed by recalibrating the model’s latent-to-physical-state affine every epoch, by least squares against the true state on training data, rather than letting it drift under gradient descent alone (Comp/PhysicsLatentAE.py:CalibrateGauge). Loss function, architecture and budget are unchanged — only how that one internal coordinate transform is determined.

	before	after
val_fcst	17.6, rising	0.015
latent → true-state probe R^2	[0.64, 0.70, 0.10]	[0.996, 0.989, 0.981]
VPT, Lyapunov times	0.130	0.378 ± 0.019

Ours still leads the (now-fixed) PINN on reconstruction, VPT and divergence. But at 50 Lyapunov times — long past where short-horizon error saturates — the PINN’s attractor fidelity is the best in the whole comparison: Wasserstein distance to the true state distribution of **0.068**, against Ours’s 0.170. Reported plainly: on that one axis, the model handed the exact equations is now the strongest tracker here, which is a legitimate result of fixing a baseline that was previously too broken to say anything about.

5.6 Model D: does bounding alone explain Ours?

	Ours	AEGRU+sigmoid (D)	ϕ sweep, 1× budget ($\phi=0$ and $\phi=1$ excluded — both degenerate by construction, §4.1)
Copy error, × floor	1.06×	2.28×	1.08×–2.01×
VPT, Lyapunov times	0.489	0.329	0.320–0.335
Divergence @ 5LT	0%	0%	100%

No, not on its own. Model D copies worse than nearly every ϕ setting and predicts no further than the ϕ ceiling, so bounding a single shared latent buys neither of Ours’s headline advantages. It does buy the *stability* half outright — 0% divergence, matching Ours, against 100% for every unbounded weighted-loss setting at the standard budget. That is the clean statement of what splitting the carriers adds on top of bounding one of them: bounding alone stops the blow-ups; separating them is what additionally protects reconstruction and extends the forecast horizon.

5.7 Noise robustness

The headline comparison (Ours, $\phi=0.1$, $\phi=0.5$, AEGRU+sigmoid, persistence, climatology), re-run at observation noise 0.15 and 0.30 in addition to the existing 0.05, one seed per model at the two new levels:

noise	Ours VPT	best ϕ VPT	gap
0.05	0.489	0.326	+0.163
0.15	0.598	0.308	+0.290
0.30	0.462	0.254	+0.208

Ours leads at every noise level tested — the gap never narrows to zero or reverses — but it is **not monotonic**: it peaks at noise 0.15 rather than moving in one direction as noise increases. Reported as the sequence rather than forced into a single “widens” or “narrows,” since that is what the numbers actually show. Ours’s divergence stays at 0% across all three noise levels; every ϕ setting’s divergence is 100% at 0.05 and 0.15 and drops only at 0.30.

5.8 A reconstruction number worth flagging

One model in the final run sits fractionally *below* the measured noise floor: $\phi=0.25$ at $10\times$ budget (§5.4), $0.99\times$ — the smallest violation in the table and the only one present in the final committed run. It is not a second data leak: Warm, the evaluation windows, are built once and reused for every row, so a leak would lift all of them, not one. A dedicated control — the same $\phi=0$ configuration retrained with $\text{latent}=3$ instead of JointAEGRU’s default 16, i.e. capacity- matched to the true Lorenz dimensionality — moved cleanly above the floor ($1.10\times$), which points at spare latent capacity (16 vs. the true 3) letting a heavily-trained, unregularised shared latent denoise slightly past the naive per-channel floor via cross-channel redundancy, rather than at a correctness bug. The effect size is small enough (under 1%) that which side of 1.0 a given run lands on varies with ordinary seed noise — an earlier run had $\phi=0$ itself ($1\times$ budget) below the floor at $0.98\times$; the current run has it at $1.01\times$, same configuration, same seed.

6. Repository

Src/	the architecture
Encoder.py	$E : x \rightarrow h$
LatentRecon.py	$f : h \rightarrow b$ (unbounded, k-dim)
LatentMapping.py	$m : b \rightarrow C$ (bounded, sigmoid)
LatentBounded.py	$r : h \rightarrow C$ (training-only teacher)
LatentForecast.py	$g : b_t \rightarrow b_{t+1}$ (residual GRU)
Decoder.py	$D : C \rightarrow x_{\text{hat}}$
DecoupledModel.py	assembly, Rollout, stage partitions
Comp/	baselines (nothing here is ours)
JointAEGRU.py	the weighted-loss knob
JointAEGRUSigmoid.py	the knob, with a bounded shared latent (§5.4)
PhysicsLatentAE.py	the PINN, RK4 on the true field
LorenzField.py	differentiable Lorenz-63 RHS
Utils/	DataLoading, Metrics, Rollout, Benchmark, Plotting, Checkpoints
SyntheticGenerators/	LorenzLift.py – the data generator, single- and multi-series
Experimentation/	
NeuralNetworkApproximation.ipynb	Stage 1 alone, in detail
LatentForecastComparison.ipynb	the single-series comparison
LatentForecastComparisonMultiSeries.ipynb	the 8-series pool + held-out generalisation test
Data/	LorenzLift.npz, LorenzEval.npz, LorenzLiftMulti.npz, LorenzLiftMulti_noise0.15.npz, LorenzLiftMulti_noise0.3.npz (§5.7)
Checkpoints/	trained model weights, cached by Utils.Checkpoints.TrainOrLoad
multi_series/	the main 8-series-pool comparison
multi_series/noise_sweep/noise{0.15,0.3}/	the two extra noise levels (§5.7)
Figures/	every figure in this README, 300 dpi

Reproducing

```
pip install -r requirements.txt
python SyntheticGenerators/LorenzLift.py          # regenerate the single-series data (optional)
jupyter lab Experimentation/LatentForecastComparison.ipynb      # single-series
jupyter lab Experimentation/LatentForecastComparisonMultiSeries.ipynb # 8-series pool + generalis
```

Set `QUICK = True` in the config cell for a fast smoke run. A full run of the multi-series notebook trains on the order of twenty model configurations (the main comparison, the seeded baselines, the budget control, the PINN diagnostics, and the noise sweep) at 120 s each; end to end, from an empty `Checkpoints/` directory, this takes roughly **three hours** on one pinned CPU thread. Every trained model is cached to `Checkpoints/` the first time it trains (`Utils/Checkpoints.py:TrainOrLoad`) — re-running the notebook after editing

an evaluation or plotting cell loads the cached weights instead of retraining; delete a checkpoint file (or the whole directory) to force that model to retrain. The noise-sweep datasets (§5.7) are generated automatically on first use if missing.

Caveats worth knowing

- **One system, one lift, one noise level in the headline comparison.** §5.7 adds two more noise levels; turbofan, weather or bearing data are still needed before this is a claim about representations in general rather than about Lorenz-63.
- **Seeds are still uneven by design, not oversight.** Ours, the PINN, $\phi=0.1/0.25/0.5$, and AE-GRU+sigmoid run 3 seeds; every other ϕ setting and the noise-sweep runs are 1 seed — extending every row to 3 seeds was judged not worth roughly tripling an already ~3-hour run for points a reviewer is unlikely to query. A VPT std of 0.0000 still means $n = 1$ on those specific rows.
- **Absolute horizons are still short.** 0.32–0.60 Lyapunov times across the models and noise levels tested is on the order of 35–65 steps. Chaos is hard; every model here fails eventually. The claim is comparative, not absolute.
- **The ϕ baseline and Model D are both constructions**, not named published methods — the weighted-loss autoencoder+GRU in its plainest form, and the same construction with a bounded shared latent.
- **The 10× budget control (§5.4) used one seed.** It answers “is 120s enough” with a clear yes-or- no; it does not by itself establish where the ϕ sweep’s ceiling would land with both 10× budget and 3 seeds together — that combination was not run.
- **A small reconstruction-floor violation persists in the final run** (§5.8): $\phi=0.25$ at 10× budget sits at 0.99× the measured floor. Investigated, attributed to spare latent capacity via a dedicated `latent=3` control, and small enough (under 1%) that it is within ordinary seed-to-seed noise for this configuration — but it is a real number in the committed table, not smoothed away.
- **Every number here is one run of a stochastic training process.** Re-running the same notebook end to end changes VPT, the ablation gap, and which side of the noise floor a given model lands on — compare the successive full extractions of this experiment in the git history for a feel for the spread, most visibly in §5.8.